

森灵小木机器人编程版 API 文档与 编程指南 1.0

目录

第一章：环境准备	4
1.1 概述	4
1.2 硬件准备	4
1.3 连接 Wi-Fi	4
1.4 获取机器人的 IP 地址	4
1.5 远程访问机器人	5
1.5.1 登录凭证	5
1.5.2 使用 SSH 登录	5
1.5.3 使用 SFTP 传输文件	6
1.6 编程环境说明	7
1.6.1 Python 版本	7
1.6.2 ROS 2 环境	7
1.6.3 工作空间结构	7
1.7 编程示例说明	8
1.8 编译与运行	9
1.8.1 编译工作空间	9
1.8.2 加载环境变量	9
1.8.3 运行示例程序	10
1.9 创建自定义程序	10
1.9.1 新增 Python 文件	10
1.9.2 新增 Launch 启动脚本	11
1.9.3 修改 setup.py 添加入口	11
1.10 重要注意事项	13
1.10.1 程序冲突处理	13
1.10.2 系统环境注意事项	13
1.10.3 系统维护提醒	13
1.11 常见问题排查	13
1.12 环境准备检查清单	14
第二章：ROS 2 消息接口与 API 参考	14
2.1 概述	14
2.2 消息文件说明	15
2.3 传感器消息（订阅）	15
2.3.1 温湿度传感器（Sensor）	16
2.3.2 IMU 传感器（Imu）	16
2.3.3 触摸传感器（Touch）	17
2.3.4 语音识别（Listen）	17
2.3.5 RFID 读卡器（Rifd）	错误!未定义书签。
2.3.6 电池状态（BatteryStatus）	18
2.3.7 视觉识别结果（Saw）	18
2.4 控制消息（发布）	19
2.4.1 语音输出（Speak）	19
2.4.2 动作控制（Action）	19

2.4.3 视觉控制 (See)	20
2.5 消息接口汇总表	20
2.6 API 速查表	21
第三章：综合应用开发	21
3.1 概述	21
3.2 开发流程	22
3.3 综合应用示例	22
3.3.1 语音输出 (`xiaomu_speak.py`)	22
3.3.2 触摸互动 (`xiaomu_touch.py`)	22
3.3.3 传感器读取 (`xiaomu_sense.py`)	23
3.3.4 动作控制 (`xiaomu_move.py`)	23
3.3.5 视觉识别 (`xiaomu_vision.py`)	23
3.3.6 智能聊天 (`xiaomu_chat.py`)	24
3.3.7 基础传感器读取 (`simple.py`)	24
3.4 示例难度梯度说明	24
3.5 多节点同时运行	25
3.6 添加自定义程序	25
3.6.1 新增 Python 文件	25
3.6.2 修改 setup.py 添加入口	25
3.6.3 创建 launch 文件	25
3.6.4 编译运行	26
3.7 开发最佳实践	26
3.8 常见问题	26
3.9 本章小结	26
附录 A：常用命令速查	27
附录 B：目录结构	27
附录 C：版权与保密声明	28
C.1 版权声明	28
C.2 保密声明	28
C.3 违反后果	29

第一章：环境准备

1.1 概述

小木机器人编程版搭载 Linux 操作系统，用户可以通过网络远程访问机器人，使用 Python 3.9 和 ROS 2 框架（Iron）进行二次开发。本章将指导您完成编程环境的准备工作。

1.2 硬件准备

在开始编程之前，请确保以下设备已就绪：

- 小木机器人（已开机）
- 路由器（支持 Wi-Fi）
- 一台可以连接网络的电脑（Windows / macOS / Linux）

1.3 连接 Wi-Fi

小木机器人需要通过 Wi-Fi 连接到局域网，以便您的电脑可以远程访问。

操作步骤：

1. 按照小木产品说明书中的步骤，启动小木机器人
2. 进入小木的网络配置模式（具体操作见产品说明书）
3. 选择您要连接的 Wi-Fi 网络并输入密码
4. 等待机器人确认连接成功

 **提示**：连接成功后，机器人会播报“网络已连接”或类似提示音。

1.4 获取机器人的 IP 地址

小木机器人的 IP 地址可以通过路由器管理界面查看。

操作步骤：

1. 在电脑浏览器中输入路由器的管理地址（通常是 `192.168.1.1` 或 `192.168.0.1`）
2. 输入路由器管理员账号密码登录
3. 找到“已连接设备”、“DHCP 客户端列表”或“设备管理”等菜单
4. 在列表中找到名为 `linaro-alip` 或类似名称的设备
5. 记录该设备的 IP 地址（例如 `192.168.1.101`）

 ****注意****：路由器管理界面因品牌型号不同而有所差异，具体请参考路由器说明书。

1.5 远程访问机器人

获取 IP 地址后，您可以使用 SSH 和 SFTP 工具远程访问机器人。

1.5.1 登录凭证

项目	信息
用户名	`linaro`
密码	`linaro`
端口	`22`（默认）

1.5.2 使用 SSH 登录

****Windows 用户：****

推荐使用以下工具：

- ****PuTTY****（轻量级 SSH 客户端）
- ****MobaXterm****（功能更全面）
- ****PowerShell****（Windows 10/11 自带）

以 PowerShell 为例：

```
""powershell
ssh linaro@<机器人 IP 地址>
""
```

****macOS / Linux 用户: ****

打开终端，执行以下命令：

```
``bash
ssh linaro@<机器人 IP 地址>
``
```

例如：

```
``bash
ssh linaro@192.168.1.100
``
```

首次连接时会提示确认指纹，输入 `yes` 即可。

1.5.3 使用 SFTP 传输文件

SFTP (SSH File Transfer Protocol) 用于在电脑和机器人之间传输文件。

****推荐的 SFTP 工具: ****

操作系统	推荐工具
Windows	WinSCP, MobaXterm, FileZilla
macOS	Cyberduck, FileZilla, 终端
Linux	Remmina, FileZilla, 终端命令

****使用终端进行 SFTP: ****

```
``bash
sftp linaro@<机器人 IP 地址>
``
```

****使用图形化工具 (以 WinSCP 为例): ****

1. 打开 WinSCP
2. 文件协议选择 `SFTP`
3. 主机名输入机器人 IP 地址
4. 用户名 `linaro`, 密码 `linaro`
5. 点击登录

1.6 编程环境说明

1.6.1 Python 版本

小木编程版支持 **Python 3.9**：

```
``bash
# SSH 登录后验证 Python 版本
python3 --version
# 输出示例：Python 3.9.2
``
```

1.6.2 ROS 2 环境

小木机器人基于 ROS 2 框架运行，用户开发时需要了解 ROS 2 的基本概念（节点、话题、消息等）。

常用 ROS 2 命令：

```
``bash
# 查看 ROS 2 节点列表
ros2 node list

# 查看话题列表
ros2 topic list

# 查看消息数据结构
ros2 interface show robot_interfaces/msg/Speak
``
```

1.6.3 工作空间结构

用户工作空间位于 `~/linaro/tm_study/`，目录结构如下：

```
``bash
/home/linaro/tm_study/
├── src/                                # 源代码目录
│   ├── example/                       # 示例功能包
│   └── example/                       # Python 模块目录
``
```

```

|         |   |—— _init_.py
|         |   |—— simple.py      # 示例：基础传感器读取
|         |   |—— xiaomu_speak.py # 示例：语音输出
|         |   |—— xiaomu_touch.py # 示例：触摸互动
|         |   |—— xiaomu_sense.py # 示例：传感器读取
|         |   |—— xiaomu_move.py  # 示例：动作控制
|         |   |—— xiaomu_chat.py  # 示例：智能聊天
|         |   |—— xiaomu_vision.py # 示例：视觉识别
|         |—— setup.py            # 包安装配置
|         |—— package.xml        # 包依赖声明
|—— launch/                      # 启动脚本目录（与 src 同级）
|   |—— simple_launch.py
|   |—— speak_launch.py
|   |—— touch_launch.py
|   |—— sense_launch.py
|   |—— move_launch.py
|   |—— chat_launch.py
|   |—— vision_launch.py
|—— install/                    # 编译安装目录（编译后自动生成）
...

```

****目录结构说明：****

```

| 目录/文件 | 用途 |
|-----|-----|
| `src/example/example/` | Python 程序文件目录 (*.py) |
| `launch/` | ROS 2 启动脚本目录 (*.launch.py) |
| `src/example/setup.py` | 包安装配置文件，定义程序入口点 |
| `src/example/package.xml` | 包依赖声明文件 |
| `install/` | 编译后的安装目录 |

```

1.7 编程示例说明

`tm\_study` 目录下提供了完整的编程示例，帮助您快速上手：

```

| 示例文件 | 功能描述 |
|-----|-----|
| `simple.py` | 基础示例：订阅传感器、触摸、IMU、视觉、语音识别等所有话题，展示数据读取 |
| `xiaomu_speak.py` | 语音输出示例：让小木说话 |
| `xiaomu_touch.py` | 触摸互动示例：触摸头部和腹部，小木做出反应 |

```


`xiaomu\_sense.py`	传感器示例：读取温湿度、IMU 数据并播报
`xiaomu\_move.py`	动作控制示例：点头、摇头、挥手等动作
`xiaomu\_chat.py`	智能聊天示例：综合语音识别、动作、触摸的完整交互
`xiaomu\_vision.py`	视觉识别示例：开启视觉，识别人脸情绪、物体并做出反应

****配套启动脚本 (launch 目录): ****

- `simple\_launch.py`
- `speak\_launch.py`
- `touch\_launch.py`
- `sense\_launch.py`
- `move\_launch.py`
- `chat\_launch.py`
- `vision\_launch.py`

1.8 编译与运行

1.8.1 编译工作空间

```
``bash
cd /home/linaro/tm_study
colcon build --packages-select example
``
```

或者直接编译所有模块：

```
``bash
cd /home/linaro/tm_study
colcon build
``
```

1.8.2 加载环境变量

```
``bash
source /home/linaro/tm_study/install/setup.bash
``
```

1.8.3 运行示例程序

****统一使用 ros2 launch 方式启动: ****

```
``bash
cd /home/linaro/tm_study

# 基础示例
ros2 launch launch/simple_launch.py

# 语音输出示例
ros2 launch launch/speak_launch.py

# 触摸互动示例
ros2 launch launch/touch_launch.py

# 传感器示例
ros2 launch launch/sense_launch.py

# 动作控制示例
ros2 launch launch/move_launch.py

# 智能聊天示例
ros2 launch launch/chat_launch.py

# 视觉识别示例
ros2 launch launch/vision_launch.py
``
```

1.9 创建自定义程序

1.9.1 新增 Python 文件

用户新增的自定义 Python 文件应放在以下目录:

```
``bash
/home/linaro/tm_study/src/example/example/
``
```

****示例：创建自定义程序****

```
``bash
# SSH 登录后执行
cd /home/linaro/tm_study/src/example/example/
touch my_custom_node.py
...

```

也可以在本地电脑上开发完成后通过 sftp 将相关文件传输到对应目录

1.9.2 新增 Launch 启动脚本

用户新增的 launch 启动脚本应放在以下目录：

```
``bash
/home/linaro/tm_study/launch/
...

```

****Launch 脚本模板： ****

```
``python
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='example',
            namespace='tmmini',
            executable='my_custom_node',
            name='my_custom_node'
        ),
    ])
...

```

1.9.3 修改 setup.py 添加入口

在 `/home/linaro/tm\_study/src/example/setup.py` 文件中，找到 `entry\_points` 部分，添加您的程序入口：

```
``python
entry_points={
    'console_scripts': [

```

```

        'simple = example.simple:main',
        'speak = example.xiaomu_speak:main',
        'touch = example.xiaomu_touch:main',
        'sense = example.xiaomu_sense:main',
        'move = example.xiaomu_move:main',
        'chat = example.xiaomu_chat:main',
        'vision = example.xiaomu_vision:main',
        'my_custom_node = example.my_custom_node:main', # 添加您的程序
    ],
},
'''

```

‘entry_points’ 的格式说明:

部分	说明
可执行名称	在 launch 文件中 ‘executable’ 字段使用的名称
模块路径	Python 模块路径, 相对于 ‘/home/linaro/tm_study/src/example/’
函数名	Python 程序中的 ‘main’ 函数名

1.9.4 修改 package.xml 添加依赖

在 ‘/home/linaro/tm_study/src/example/package.xml’ 文件中, 添加第三方 Python 依赖:

```

'''xml
<!-- 添加 Python 依赖 -->
<exec_depend>python3-requests</exec_depend>
<exec_depend>python3-numpy</exec_depend>
<exec_depend>python3-opencv</exec_depend>
'''

```

****常用依赖说明: ****

依赖名称	用途
python3-numpy	数值计算
python3-opencv	图像处理
python3-requests	HTTP 请求

 ****提示**:** 添加依赖后, 需要重新编译工作空间才能生效。

1.10 重要注意事项

1.10.1 程序冲突处理

用户在编写自己的程序时，可能与机器人主控程序产生资源冲突（例如占用相同的传感器或执行器），导致机器人主控运行异常。

****解决方法：****

- 1. ****停止用户自定义程序****：在终端中按 `Ctrl + C` 终止正在运行的用户程序
- 2. ****重启机器人主控****：重启机器人即可恢复

 ****建议****：开发调试时，在自己的程序中避免占用主控必需的关键资源。

1.10.2 系统环境注意事项

如果您对 Linux 系统不够熟悉，请遵守以下原则：

-  ****不要随意修改系统参数****
-  ****不要修改系统环境变量****
-  ****不要删除或修改系统关键目录****
-  ****只在自己的工作目录下操作****（`/home/linaro/tm\_study/`）
-  ****只通过 SSH/SFTP 进行开发和文件传输****

1.10.3 系统维护提醒

- ****请勿尝试攻击机器人系统****：包括但不限于破解系统权限、恶意修改系统文件等行为
- ****请勿删除系统关键文件****
- ****共同维护良好的开发环境****，感谢您的配合与理解

1.11 常见问题排查

问题	可能原因	解决方法
-----	-----	-----

SSH 连接失败	IP 地址错误或网络不通	重新通过路由器确认 IP 地址，检查机器人和电脑是否在同一网络
密码错误	大写锁定或输入错误	确认密码为 `linaro` (小写)，检查键盘状态
SFTP 传输失败	磁盘空间不足	执行 `df -h` 查看磁盘空间，清理不必要的文件
程序运行报错	Python 依赖缺失	在 `package.xml` 中添加依赖并重新编译
编译失败	setup.py 配置错误	检查 `entry\_points` 格式是否正确
Launch 文件找不到	launch 文件未正确放置或未配置	确认 launch 文件位于 `/home/linaro/tm\_study/launch/` 目录
与主控冲突	资源占用冲突	停止自定义程序或重启机器人
找不到机器人 IP	路由器未显示设备	确认机器人已连接 Wi-Fi，刷新路由器页面

1.12 环境准备检查清单

完成本章内容后，请确认以下项目：

- [] 机器人已开机并连接 Wi-Fi
- [] 已通过路由器管理界面获取机器人的 IP 地址
- [] 能够通过 SSH 成功登录机器人（用户名/密码：linaro/linaro）
- [] 能够通过 SFTP 向机器人传输文件
- [] 已确认 Python 3.9 环境正常
- [] 已了解 ROS 2 基本概念
- [] 已了解工作空间目录结构（`/home/linaro/tm\_study/`）
- [] 已了解 Python 文件存放位置（`src/example/example/`）
- [] 已了解 Launch 文件存放位置（`/home/linaro/tm\_study/launch/`）
- [] 已了解如何添加自定义程序入口（`setup.py`）
- [] 已了解如何添加第三方依赖（`package.xml`）
- [] 已了解如何编译和运行程序
- [] 已熟悉提供的编程示例

第二章：ROS 2 消息接口与 API 参考

2.1 概述

小木机器人的所有功能模块通过 ROS 2 消息接口进行通信。用户通过订阅和发布特定话题，

即可获取传感器数据、控制机器人动作、输出语音等。

本章介绍小木机器人提供的消息接口。****消息的详细定义（包括字段类型、常量值等）请以`ros2 interface show` 命令查询的结果为准。****

```
> 💡 **查询消息定义的方法**:  
> ```bash  
> # 查看所有可用消息  
> ros2 interface list | grep robot_interfaces  
>  
> # 查看具体消息的定义  
> ros2 interface show robot_interfaces/msg/Speak  
> ros2 interface show robot_interfaces/msg/Touch  
> ros2 interface show robot_interfaces/msg/Action  
> ```  
  
---
```

2.2 消息文件说明

小木机器人使用自定义消息包`robot\_interfaces`,使用时需要在 Python 脚本中导入对应的消息类型:

```
```python  
from robot_interfaces.msg import Sensor, Imu, Touch, Speak, Listen, Action, Servo, See, Saw,
Emotion, Led, Rfid, BatteryStatus
```
```

****消息包依赖****: 在`package.xml` 中需要添加:

```
```xml  
<depend>robot_interfaces</depend>
```  
  
---
```

2.3 传感器消息（订阅）

以下消息通过订阅对应话题获取传感器数据。

| 模块 | 话题名称 | 消息类型 | 说明 |
|----|------|------|----|
|----|------|------|----|

```
|-----|-----|-----|-----|
| 温湿度 | `/sensor` | `Sensor` | 获取温湿度数据 |
| IMU | `/imu` | `Imu` | 获取姿态和运动数据 |
| 触摸 | `/touch` | `Touch` | 获取触摸状态 |
| 语音识别 | `/listen` | `Listen` | 获取语音识别文本 |
| 电池 | `/battery` | `BatteryStatus` | 获取电池状态 |
| 视觉结果 | `/saw` | `Saw` | 获取视觉识别结果 |
```

2.3.1 温湿度传感器 (Sensor)

****话题**:** `/sensor`

****使用示例**:**

```
```python
def sensor_callback(self, msg):
 temp = msg.temperature
 humidity = msg.humidity
 self.get_logger().info(f'温度: {temp:.1f}°C, 湿度: {humidity:.1f}%')

 if temp > 35:
 self._speak("好热啊，快帮我降降温吧")
...

```

## 2.3.2 IMU 传感器 (Imu)

**\*\*话题\*\*:** `/imu`

**\*\*使用示例\*\*:**

```
```python
import math

def imu_callback(self, msg):
    ax, ay, az = msg.accel_x, msg.accel_y, msg.accel_z
    pitch = math.atan2(ax, az) * 180 / math.pi
    self.get_logger().info(f'俯仰角: {pitch:.1f}°')
...

---
```

2.3.3 触摸传感器 (Touch)

****话题**:** `/touch``

****说明**:** ``head_touch_state`` 和 ``belly_touch_state`` 字段值为 0 表示无触摸, 1 表示有触摸。

****使用示例**:**

```
``python
def touch_callback(self, msg):
    if msg.head_touch_state == 1:
        self._speak("嘿嘿, 摸头好舒服呀")
        self._nod()

    if msg.belly_touch_state == 1:
        self._speak("嘻嘻, 好痒啊")
        self._shake_head()
...

```

2.3.4 语音识别 (Listen)

****话题**:** `/listen``

****使用示例**:**

```
``python
def listen_callback(self, msg):
    text = msg.data.lower()

    if '你好' in text:
        self._speak("你好! 很高兴见到你")
        self._wave()
    elif '再见' in text:
        self._speak("再见, 下次见")
        self._wave()
...

```

2.3.5 电池状态 (BatteryStatus)

****话题**:** `/battery`

****使用示例**:**

```
```python
def battery_callback(self, msg):
 if msg.status == 1: # 具体常量值请通过 ros2 interface show 查看
 self.get_logger().info('正在充电')
 elif msg.status == 2:
 self.get_logger().info('电池已充满')
 else:
 self.get_logger().info(f'当前电量: {msg.percentage}%%')
...
```
```

2.3.6 视觉识别结果 (Saw)

****话题**:** `/saw`

****说明**:** 消息包含 `persons` 和 `things` 两个列表，分别存储检测到的人员和物体信息。

****使用示例**:**

```
```python
def saw_callback(self, saw_data):
 for person in saw_data.persons:
 if person.emotion == 1: # 具体常量值请通过 ros2 interface show 查看
 self._wave()
 self._speak("你看起来很开心! ")

 for thing in saw_data.things:
 if thing.type == "cat":
 self._speak("好可爱的小猫咪呀")
...
```
```

2.4 控制消息（发布）

以下消息通过发布到对应话题来控制机器人。

| 模块 | 话题名称 | 消息类型 | 说明 |
|------|-----------------------|-----------------------|-----------|
| 语音输出 | <code>/speak`</code>  | <code>`Speak`</code> | 播报文本 |
| 动作控制 | <code>/action`</code> | <code>`Action`</code> | 控制舵机和 LED |
| 视觉控制 | <code>/eye`</code>    | <code>`See`</code> | 开启/关闭视觉识别 |

2.4.1 语音输出（Speak）

****话题**:** `/speak``

****使用示例**:**

```
```python
def _speak(self, text):
 msg = Speak()
 msg.text = text
 self.speak_pub.publish(msg)
```
```

注意：输出的文字必须和机器人当前的系统语言一致，否则会播报异常。

2.4.2 动作控制（Action）

****话题**:** `/action``

****说明**:**

- ``Action`` 消息包含 ``time``（执行时间）、``type``（动作类型）、``servos``（舵机列表）、``leds``（LED 列表）
- 动作类型含义请通过 ``ros2 interface show`` 查看常量定义

****使用示例**:**

```
```python
```

```

def _wave(self):
 action = Action()
 action.time = 0.5
 action.type = 0 # 具体常量值请通过 ros2 interface show 查看

 servo = Servo()
 servo.index = 4 # 舵机索引含义请通过 ros2 interface show 查看
 servo.angle = 45
 action.servos.append(servo)

 self.action_pub.publish(action)
...

```python
def reset(self, duration=1.0):
    action = Action()
    action.time = float(duration)
    action.type = 2 # 复位类型
    self.action_pub.publish(action)
...

---

```

2.4.3 视觉控制 (See)

****话题**:** `/eye``

****使用示例**:**

```

```python
def start_vision(self):
 see = See()
 see.command = 1 # 具体常量值请通过 ros2 interface show 查看
 self.eye_pub.publish(see)
 self._speak("视觉识别已开启")
...

```

## 2.5 消息接口汇总表

模块	话题名称	消息类型	方向	说明
----	------	------	----	----

功能	操作	接口
温湿度	订阅	获取温湿度数据
IMU	订阅	获取姿态和运动数据
触摸	订阅	获取触摸状态 (0/1)
语音识别	订阅	获取识别文本
电池	订阅	获取电池状态
视觉结果	订阅	获取视觉识别结果
语音输出	发布	播报文本
动作控制	发布	控制舵机和 LED
视觉控制	发布	开启/关闭视觉

## 2.6 API 速查表

功能	操作	接口
让小木说话	发布	`Speak`
控制舵机	发布	`Action` + `Servo`
复位舵机	发布	`Action` (复位类型)
开启视觉	发布	`See` (start 命令)
读取温湿度	订阅	`Sensor`
读取姿态	订阅	`Imu`
读取触摸	订阅	`Touch`
读取语音	订阅	`Listen`
获取视觉结果	订阅	`Saw`

> 💡 \*\*重要提示\*\*：消息中的字段类型、常量值、舵机索引等请以 `ros2 interface show` 命令的实际输出为准。

## 第三章：综合应用开发

### 3.1 概述

掌握了第二章的消息接口后，本章将指导您综合运用各模块，开发具有实际功能的机器人应用。通过组合使用语音、动作、传感器、视觉等能力，您可以创造出丰富的交互体验。

---

## 3.2 开发流程

开发一个完整的机器人应用通常遵循以下步骤：

1. **确定功能需求**：明确机器人需要响应的场景和行为
2. **设计交互逻辑**：规划触发条件、响应动作、状态转换
3. **编写节点代码**：实现订阅和发布逻辑
4. **添加启动脚本**：创建 launch 文件便于启动
5. **编译和测试**：编译工作空间，运行验证

---

## 3.3 综合应用示例

以下示例基于 `tm\_study/src/example/example/` 目录中已有的示例文件，从简单到复杂排列，您可以根据自己的需求选择学习和参考。

### 3.3.1 语音输出（`xiaomu\_speak.py`）

**功能**：让小木说话的基础示例，演示最基础的语音输出能力。

**涉及接口**：`Speak`（语音输出）

**启动命令**：

```
``bash
ros2 launch launch/speak_launch.py
``
```

---

### 3.3.2 触摸互动（`xiaomu\_touch.py`）

**功能**：触摸头部和腹部时，小木做出语音和动作反应。

**涉及接口**：`Touch`（触摸检测）、`Speak`（语音输出）、`Action`（动作控制）

**\*\*启动命令\*\*:**

```
``bash
ros2 launch launch/touch_launch.py
``
```

---

### 3.3.3 传感器读取 (`xiaomu\_sense.py`)

**\*\*功能\*\*:** 读取温湿度和 IMU 传感器数据并播报。

**\*\*涉及接口\*\*:** `Sensor` (温湿度)、`Imu` (姿态传感器)、`Speak` (语音输出)

**\*\*启动命令\*\*:**

```
``bash
ros2 launch launch/sense_launch.py
``
```

---

### 3.3.4 动作控制 (`xiaomu\_move.py`)

**\*\*功能\*\*:** 控制小木做各种动作，包括点头、摇头、挥手、复位等。

**\*\*涉及接口\*\*:** `Action` (动作控制)、`Speak` (语音反馈)

**\*\*启动命令\*\*:**

```
``bash
ros2 launch launch/move_launch.py
``
```

---

### 3.3.5 视觉识别 (`xiaomu\_vision.py`)

**\*\*功能\*\*:** 开启视觉识别，识别人脸情绪和物体并做出反应。

**\*\*涉及接口\*\*:** `See` (视觉控制)、`Saw` (视觉结果)、`Speak` (语音输出)、`Action` (动作

控制)

**\*\*启动命令\*\*:**

```
``bash
ros2 launch launch/vision_launch.py
``
```

---

### 3.3.6 智能聊天 (`xiaomu\_chat.py`)

**\*\*功能\*\*:** 综合语音识别、动作控制、触摸互动的完整交互示例。

**\*\*涉及接口\*\*:** `Listen` (语音识别)、`Touch` (触摸检测)、`Speak` (语音输出)、`Action` (动作控制)

**\*\*启动命令\*\*:**

```
``bash
ros2 launch launch/chat_launch.py
``
```

---

### 3.3.7 基础传感器读取 (`simple.py`)

**\*\*功能\*\*:** 订阅所有传感器话题, 展示数据读取方法, 适合作为开发自定义节点的参考模板。

**\*\*涉及接口\*\*:** `Sensor`、`Imu`、`Touch`、`Listen`、`Saw`、`Rifd`、`BatteryStatus`

**\*\*启动命令\*\*:**

```
``bash
ros2 launch launch/simple_launch.py
``
```

---

## 3.4 示例难度梯度说明

| 难度 | 示例 | 核心学习目标 |



```
|-----|-----|-----|
| ★☆☆ | `xiaomu_speak.py` | 掌握消息发布 (Speak) |
| ★☆☆ | `xiaomu_touch.py` | 掌握消息订阅 (Touch) + 简单响应 |
| ★★☆☆ | `xiaomu_sense.py` | 掌握传感器数据处理 |
| ★★☆☆ | `xiaomu_move.py` | 掌握动作控制 (Action + Servo) |
| ★★★★★ | `xiaomu_vision.py` | 掌握视觉模块控制与结果处理 |
| ★★★★★ | `xiaomu_chat.py` | 综合运用多模块实现完整交互 |
| ★★★★★ | `simple.py` | 了解所有消息接口, 作为开发模板 |
```

---

## 3.5 多节点同时运行

可以使用一个 `launch` 文件同时启动多个节点, 参考 ``launch`` 目录下的现有文件格式。

---

## 3.6 添加自定义程序

### 3.6.1 新增 Python 文件

在 ``/home/linaro/tm_study/src/example/example/`` 目录下创建新的 ``.py`` 文件, 参考现有示例的结构编写代码。

### 3.6.2 修改 `setup.py` 添加入口

在 ``.setup.py`` 的 ``.entry_points`` 中添加新程序的入口:

```
```python
'my_node = example.my_node:main',
```
```

### 3.6.3 创建 `launch` 文件

在 ``/home/linaro/tm_study/launch/`` 目录下创建对应的 ``.launch.py`` 文件。

## 3.6.4 编译运行

```
``bash
cd /home/linaro/tm_study
colcon build --packages-select example
source install/setup.bash
ros2 launch launch/my_node_launch.py
``
```

## 3.7 开发最佳实践

| 实践   | 说明                                                                    |
|------|-----------------------------------------------------------------------|
| 单一职责 | 每个节点只负责一个明确的功能模块                                                      |
| 松耦合  | 节点之间通过话题通信，避免直接依赖                                                     |
| 参数化  | 使用 <code>declare_parameter</code> 和 <code>get_parameter</code> 使节点可配置 |
| 日志分级 | 使用 <code>debug/info/warn/error</code> 不同级别记录日志                        |
| 异常处理 | 捕获异常避免节点崩溃                                                            |

## 3.8 常见问题

| 问题       | 解决方法                                                                                      |
|----------|-------------------------------------------------------------------------------------------|
| 节点启动后无响应 | 执行 <code>source install/setup.bash</code> 加载环境                                            |
| 话题收不到数据  | 检查 <code>launch</code> 文件中的 <code>namespace</code> 设置是否为 <code>tmmini</code>              |
| 编译找不到消息  | 确认 <code>package.xml</code> 中有 <code>&lt;depend&gt;robot_interfaces&lt;/depend&gt;</code> |
| 多个节点冲突   | 停止其他节点或使用不同的命名空间                                                                          |

## 3.9 本章小结

`tm_study` 工作空间中提供了从简单到复杂的编程示例，涵盖语音、动作、传感器、触摸、视觉等各个模块。建议按以下路径学习：

1. 从 `xiaomu\_speak.py` 开始，了解最基础的发布-订阅模型
2. 依次学习触摸、传感器、动作控制示例
3. 尝试视觉识别和智能聊天等综合示例
4. 参考 `simple.py` 了解所有接口，开始开发自己的节点

各示例对应的启动脚本位于 `/home/linaro/tm\_study/launch/` 目录，与示例文件一一对应。

---

## 附录 A：常用命令速查

```
``bash
Ros2 环境设置(本命令在 ssh 登录时默认会执行)
Source /opt/ros2/iron/setup.bash

项目环境设置
source /home/linaro/tm_study/install/setup.bash

编译
cd /home/linaro/tm_study && colcon build --packages-select example

查看话题
ros2 topic list

查看消息定义
ros2 interface show robot_interfaces/msg/消息名

启动示例
ros2 launch example 示例名_launch.py
``
```

## 附录 B：目录结构

```
``bash
/home/linaro/tm_study/
├── src/example/example/ # Python 程序目录
```

```
| |—— simple.py
| |—— xiaomu_speak.py
| |—— xiaomu_touch.py
| |—— xiaomu_sense.py
| |—— xiaomu_move.py
| |—— xiaomu_chat.py
| |—— xiaomu_vision.py
|—— launch/ # Launch 启动脚本目录
| |—— simple_launch.py
| |—— speak_launch.py
| |—— touch_launch.py
| |—— sense_launch.py
| |—— move_launch.py
| |—— chat_launch.py
| |—— vision_launch.py
|—— install/ # 编译输出目录
...
```

## 附录 C：版权与保密声明

### C.1 版权声明

本文档及其中包含的所有内容（包括但不限于文字、图片、代码示例、接口定义等）的版权归**梯目科技（武汉）有限公司**所有。

未经梯目科技书面许可，任何单位或个人不得：

- 复制、转载、传播本文档的全部或部分内容
- 将本文档内容公开发布到互联网、论坛、社交媒体等公开渠道
- 将本文档用于商业目的

### C.2 保密声明

本文档包含与梯目科技产品相关的技术信息、通信接口定义和开发方法，属于梯目科技的**保密信息**。

您作为小木机器人编程版的使用者，承诺：

1. **不传播**：不将本文档及其中包含的技术信息以任何形式提供给第三方
2. **不公开**：不在互联网、GitHub、技术论坛等公开渠道发布本文档内容或基于本文档的技术实现
3. **内部使用**：本文档仅限您本人或您所在团队内部用于小木机器人的二次开发

## C.3 违反后果

未经授权传播或公开本档内容的行为,可能构成侵权。梯目科技保留追究法律责任的权利。  
感谢您的理解与配合。如有疑问,请联系梯目科技官方支持。